A Modern SWE Internship Guide

By: Anonymous Reddit User

Introduction

This is basically a guide that compiles my experience of what has worked and hasn't worked for me when landing software engineer interviews and internships. It includes all the mistakes I have made, mistakes I have seen other people make, and what I wish I did in the beginning.

Who am I?

As of January 2025, I'm a Junior at a Top 100 public university. I'm your regular middle class asian male. I have received multiple internship offers during my academic career as seen below:

Summer 2024 (Sophomore - 3.9 GPA) Internship Offers

- A small no-name startup

Summer 2025 (Junior - 3.9 GPA) Internship Offers

- Microsoft - SWE Intern (accepted)
- Capital One - Technology Internship Program (TIP)
- JPMorgan Chase - SWE Intern
- Walmart Global Tech - SWE Intern
- Samsung - SWE Intern
- USAA - SWE Intern
- Some other smaller ones

View my full Summer 2025 internship search diagram here

NOTE: None of my offers came from any form of nepotism

I have also received multiple interviews and online assessments (OA) during this time as well. I will break these down later in the guide.

Of course it is up to you to believe whether or not I actually received these offers, but understand I gain nothing from lying on this anonymous guide. I just enjoy helping out future software engineers like myself.

View my anonymized resume https://imgur.com/fvIaKTV

⚠️Disclaimers⚠️

- If you do not like programming, this guide is not for you
- If you want to be a software engineer solely for the money, this guide is not for you
- If you lack passion for this field, this guide is not for you
- If you don't want to put in the effort, this guide is not for you
- Basically, if you lack ambition to become a software engineer and aren't willing to put in high amounts of effort, it will be very difficult for you to become a software engineer in this modern tech era
- Also, due to the nature of redditors, I will not be sharing any personal information in this guide; even this email is a fake name burner
- This may not be the best guide for international students either since I don't completely understand their circumstances and can't provide adequate advice

Who is this guide for?

- US Citizens
- University students - specifically undergraduates (i.e. freshman, sophomore, and junior)
- Aspiring software engineers with a passion for the field
- This guide isn't really for anyone in their last year of university

How to use this guide

Based on the class level you are a part of, go to the corresponding section and use the advice there. It also makes sense to look at the advice for the next year to prepare yourself accordingly. There are also sections of advice independent of university level so be sure to check those out as well.

General

This section is for what to do and not do during every year in university. Make sure you are doing these along with the advice in the specific university year sections.

DOs

- Stay on top of your coursework getting as close to a 4.0 GPA as you can
    - Contrary to common coping belief, GPA does matter since recruiters use it as a filtering parameter (i.e. reject all applicants with a GPA lower than a 3.0)

- Always be working on some sort of personal full-stack project
- Attend recruiting/networking events whenever possible to build crucial communication skills and network with professionals in the field
- Stay up to date on tech news (i.e. internship releases, new technologies, etc.)
    - I personally followed some instagram SWE influencers for this like @zero2sudo
- Frequently look over your resume, proofread, and improve it where possible
- Join 1-2 coding clubs or organizations to get opportunities to work on group projects which is good for the resume and experience
- Attend hackathons for group work and experience
    - Some companies offer internships from hackathons. I personally got my JPMorgan Chase offer from performing well at their Code For Good hackathon
- Set up linkedin job alerts for job titles related to software engineer (i.e. “software engineer”, “software developer”, etc.)
- Follow internship githubs like Simplify to stay up-to-date on new SWE internship drops
- Build a publicly-hosted portfolio website showcasing your fields of interest, experience, and most importantly your projects
- Complete OAs as soon as you get them, there is no point in postponing it for multiple days
    - Just do them as soon as you have time to do them
    - Sometimes companies move people on to the next stage the sooner they complete the OA and reject people that take too long to start the OA

DONTs

- Taking an online bootcamp to replace university
- Avoid recruiting events because they are a “waste of time”
- Slack off in your courses
- Take shortcuts (i.e. just watching Harvard CS50 and thinking that is enough)

Freshman

It is going to be very difficult to land an internship your freshman year, so don’t feel too down if you don’t get one. I personally didn’t get one and I turned out just fine.

DOs

- Get good grades and pay attention to your courses. This is the foundation of your career, you don’t want to mess it up and make it difficult for you in the future.
- Choose a major either Computer Science or Computer Engineering
- Apply to the freshman/sophomore internship programs from large companies (UberSTAR, Google Step, Meta University, etc.)
    - You don’t want to miss out on these like I did
    - Stay up to date by following instagram SWE influencers

- Apply and land a Computer Science-related research position at your university
    - Since landing an internship will be hard, try really hard to get a research position instead
    - Reach out to your CS professors, CS department heads, etc
    - This will really help you in the future and is something I wish I did as a freshman
    - These research opportunities are done during the school year so make sure to find and apply to them as early as possible
- Create your resume
- Follow the general advice

DONTs

- Waste time applying to a bunch of unnamed companies

Key Points

- Land a research position at the very least
- Choose Computer Science or Computer Engineering
- Get good grades
- Join coding clubs
- Make projects

Sophomore

This year is a crucial year and will set you up for the junior year internship season which is where companies mostly hire from. I barely landed a small start up internship this year by pure luck because I slacked off during the first semester. Hopefully you will be better than me.

DOs

- Consistently be leetcoding at least 4 times a week
- Master the STAR method by talking to a wall or practice on friends
- Start applying to SWE internships like crazy
    - In my junior year I averaged 30-100 applications a day and I wish I did this in my sophomore year so I wouldn't have needed to suffer so bad
    - Apply to as many SWE internships as possible; leave no stone unturned
- Keep following the general advice

DONTs

- Stop applying until you land an offer

        - After you land an offer you can choose to continue applying for something better or just stop, relax, and be happy

Key Points

- Land an internship for the summer
    - This is a crucial goal for sophomore year
        - I would even go as far as to say to sacrifice going to some classes to attend recruiting events
- Continue to advance your resume with more projects and more personal experience

Junior

This year is the single most important year for your future software engineer career since the internship you land this year will most likely be your full-time entry job through a return offer. If you haven't landed an internship by Junior year, then you need to start taking it very seriously this year.

DOs

- Consistently be leetcoding at least 6 times a week
    - The more the better of course
- **APPLY APPLY APPLY APPLY APPLY**
    - Apply to every single SWE internship you see; leave no stone unturned
- Master the STAR method if haven't already
- Keep following the general advice

DONTs

- Stop applying until you land an offer

Key Points

- Don't stop applying until you land an offer
    - After you land an offer you can choose to continue applying for something better or just stop, relax, and be happy
- Do everything you can to get an internship
    - I personally missed 4 days of school to attend recruiting events from 9am to 5pm each day. I landed my Samsung offer and an interview with Raytheon with this.

- Apply every day so you can be a part of the first pool of applicants to an internship opportunity
- Land that crucial internship for the summer

Senior

If you have done everything right then you should have at least one internship in your belt and lots of personal projects and experience on your resume. If you have a return offer already then you are pretty set, congratulations on becoming a software engineer! If you haven't received your return offer yet or will not be receiving one at all, get to applying like crazy again. Since you should hopefully have work experience by now, getting a new offer won't be as difficult as before. HOWEVER… If you haven't been able to land an internship or a research position this entire time… It's time to apply to McDonalds. Like seriously. It's over for you. No one will hire you in this modern tech era with no experience. Choose a new career path. GG buddy.

I'm jk… kinda

If you are a Senior and have no experience yet, it isn't impossible to land a career in SWE but it will definitely be very very very difficult. The best choices you have would be getting a SWE role through a connection, aiming for a tech-related role at a smaller company and eventually move up to SWE (i.e. help desk assistant, IT assistant, etc), or just keep on applying while learning and creating projects to buff up your resume some more. Another thing you could do is do freelance work on Upwork or Fiverr or similar to get some professional-ish work on your resume. You could make people websites or something. If you are passionate about SWE then keep pursuing it no matter what. It's a hobby first and career second. Just stay diligent and go for every opportunity you can. You never know what will happen.

Summer

Something I did to make coursework much more manageable during my academic semesters was by taking many non-swe related courses at online community college during the summer. This meant I would be spending less money on college tuition for classes that don't really matter while giving myself more time to work on projects, apply more, etc. Plus online community college courses are much easier. If you have an internship to attend during the summer then obviously focus on that first. If you don't, then take some useless required college courses online while working on personal projects. This way you are always making progress toward your career little by little. Make sure to still leave yourself some time to enjoy the summer break and not burnout.

Resume

Use Jake's Resume template and use my anonymized resume for inspiration on wording. Use a LaTeX editor to create your resume. I personally like bolded keywords; some people think this doesn't make a difference but I did it anyway. Constantly keep improving your resume. Ask people around you that have landed internships for advice. Follow some youtube videos for advice as well. I attended some resume review sessions which I felt was helpful. After creating a project, completing an internship, finishing a hackathon, or something of those natures, add them to your resume to improve it even more.

Applying

After setting up linkedin job alerts and following githubs like Simplify, I would just apply online through the link provided by these sources. For any short answer questions in the applications, I would use AI to write them. I also created my own cover letter template where all I needed to do was change the company name to make it work for any job application. I just used some SWE cover letter templates online as inspiration. I have tried reaching out to people on linkedin that work at the job I'm applying for to try and get a referral but I found this to be ineffective and inefficient. Just cold apply and move on quickly.

Here is how I streamlined my application process:

1. Create a folder for bookmarks in your preferred browser
    a. This will be for opening bookmarked tabs you'll need to use during a session of applying
2. The tabs I would have would be ChatGPT, AI Humanizer, LinkedIn Jobs Picked for Me, my resume pdf, Internship GitHubs, Cover Letter Template, my email dashboard
3. I would open a new browser window for loading all the tabs of the jobs I will be applying for in this session. I would get these jobs from LinkedIn Jobs Picked for Me and internship GitHubs.
4. Then I fill out each application as quickly as I can, verifying my email with my email dashboard tab when needed, inputting a cover letter when asked, using ChatGPT + an AI Humanizer to write any short answer responses. Make sure to also input your resume into ChatGPT when asking to write short answer responses so that it can make results specific to you.
5. Turn on some focus music and get to grinding

Leetcode

Start off slowly doing easy problems and work your way to hard problems. I suggest following Neetcode's roadmap and his youtube videos to learn DSA. There are many resources online on how to leetcode and start off. Once you learn the basics, I would recommend choosing leetcodes in the sections of topics you struggle with (i.e. sliding window, dynamic programming, depth-first search, etc)

Interviews

Behavioral

- STAR method can be used on many situation-based questions
- Don't ramble on and on about things that don't matter
    - Make sure to think "am I going off topic" and then quickly trace back to the main question if you are going off topic
- When asking interviewers questions of your own, ask them things related to what they do, what you'll be doing, and some culture questions
    - When asking culture questions, don't ask just "what is company culture like"; this is so lame; ask something like "what does the company do for reaching big milestones"
    - Doing research on the role and company before the interview can help you ask more targeted questions as well
- Take it slow; you don't need to answer as quickly as possible; asking for a short couple of seconds to think (i.e. "that's a great question, let me think about it for a second")
- Take short pauses between your sentences instead of saying filler words (i.e. um, ah, etc)
- Nod your head like you are listening when they are talking
- Don't cut them off
- Look interested
- Have a happy/bright facial expression; try not to look bored

Technical

- Voice all your thoughts; like anything you are thinking just think out loud
- Ask for hints or assistance into the right direction; they want you to pass (usually)
- Don't ask if you are on the right direction
- Voice your thoughts some more; they want to see how you problem solve and think of solutions
- Whenever they offer advice or a direction to go, you need to actually use the advice
    - Don't just ignore it and do a different method even if you think your method is better
    - They want to see how you implement criticism
- Also voice time complexities of components of your solution as you create it (i.e. "going to be using a for loop but hmmm that'll make it O(n) now…")
- Using filler words is fine during this section
    - In moderation, not too much
- There may be a coding question that you simply just can't do; the only thing you can do is keep asking for help until they either fail you or you figure it out; either way you need to keep your head high and study more leetcode if you think you didn't do well.

Projects

Throughout your academic career you need to be creating projects since these are the closest thing to experience you'll have before receiving an internship. It is the only thing recruiters can go off of. I would recommend full-stack projects over other ones because it allows you to learn development from front-to-back rather than a specialized area. Recruiters want to see you know a lot about a lot rather than know a lot about a little. Hopefully that makes sense. Doing it this way broadens your tool belt and gives you more opportunities.

Choosing a Project and Technologies

You'll want to create projects going from easy to complex to allow for optimal learning and growth. Don't try to jump to the hardest sounding project with a bunch of technologies in the beginning, it'll just demotivate you due to the lack of knowledge. The technology you use in your projects should be popular industry-standard. Different projects you create should use different technology you haven't used before. This will broaden your toolset and allow you to constantly keep learning. For example, if project 1 uses React and Node + Express, then project 2 would use React and Flask. After your 2nd or 3rd project I recommend sprinkling in aspects of machine learning and AI usage. This could mean using OpenAI's api or having some machine learning model used in your project. I also recommend having cloud usage in some of your projects like with GCP, AWS, or Azure. I have listed some technologies below for different areas of development below:

Frontend

- HTML + CSS
- React/React Native
- Vue
- Svelte
- Angular

Backend

- Flask
- Node + Express
- Django
- Ruby on Rails
- .NET
- Spring Boot

Databases

- PostgreSQL
- MySQL
- MongoDB

- Firebase

Cloud

- AWS
- Google Cloud Platform (GCP)
- Azure

Based on your knowledge of building projects, you should choose a project corresponding to the level of difficulty. You should just ask ChatGPT to generate you some project ideas based on level as well. Your project doesn't need to be insanely unique, it doesn't need to be the next biggest thing, it just needs to have learning potential. Since ChatGPT can generate your project ideas, I have only listed some basic-level projects that I would recommend for your first time.

Basic

- To-do list (a classic)
- Weather dashboard
- Tic Tac Toe game
    - That keeps the score from past games in a database

Creating the Project

Now that you have chosen what project and technologies you want to use, you need to actually create the project. Here is how I would recommend doing it. If this is one of your first projects, use a lot of ChatGPT to help you UNDERSTAND how to create certain parts of the project. Don't ask it to create code for you to just copy and paste. You have to ask it to explain why it codes in a certain way and learn how it came up with the code. Use it like a mentor by your side and just ask it a bunch of questions. You should also put your code in GitHub and to practice using version control while you develop. It also allows your code to be public which some hiring managers will look at.

DOs

- Watch YouTube videos on how to use certain technology
- Use ChatGPT to help you understand how to use technology
- Use ChatGPT to create a roadmap on creating your project
- Use ChatGPT for guidance on your project like what to do next or what order to do things
- Read documentation for specific components of a technology you may need
- Use GitHub

DONTs

- Use ChatGPT to write large blocks of code for you to just copy and paste

- Use long (2 hours+) YouTube video tutorials on making a full project
    - There are so many problems with this because the video could be outdated and following step by step may not work due to version mismatches. This means after 4 hours of following the video something doesn't work and you can't fix it since you have no idea what you're doing and just following the steps

Networking

The only networking I ever did was by getting the LinkedIns of people I talked to at any recruiting event. After talking with them I would say: "I would love to connect with you after this, do you have LinkedIn?" Then after the event I would connect with them as soon as possible and send a message like "I really enjoyed meeting you at <event> on <date> and talking with you about <whatever y'all talked about>." Then lead this into trying to get a referral for their job opening. A quick google search can give me a message template for asking for a referral after a career fair. That's what I did.

For career fairs, get there early, talk to the companies you want and talk to as many as you can. The more you talk, the more chances you have at getting a referral or interview. As said before, I got my Samsung interview from this which led to an offer. I also received an interview with RTX from the career fair as well. You never know who you might meet and connect with. Just go out and do it. This type of thing can only be done in college so you don't want to regret not going.

Again, it is very important that you connect with them as soon as possible because the longer you wait, the less they remember who you are.

Hackathons

Just do your best, work together with your team, and cooperate. Some technologies I recommend for hackathons are React, Flask, and PostgreSQL. For deployment use Render.com and Supabase for PostgreSQL. These are the technologies I used for multiple hackathons and have worked well. Easy to learn and easy to set up. If it is a hackathon that gives out internships, just make sure to show mentors (if there are any) that you are trying to put in effort and seek to learn. If there are no internships being given out, just try and make something cool to potentially win. Even if you don't win, it is still good resume fuel.

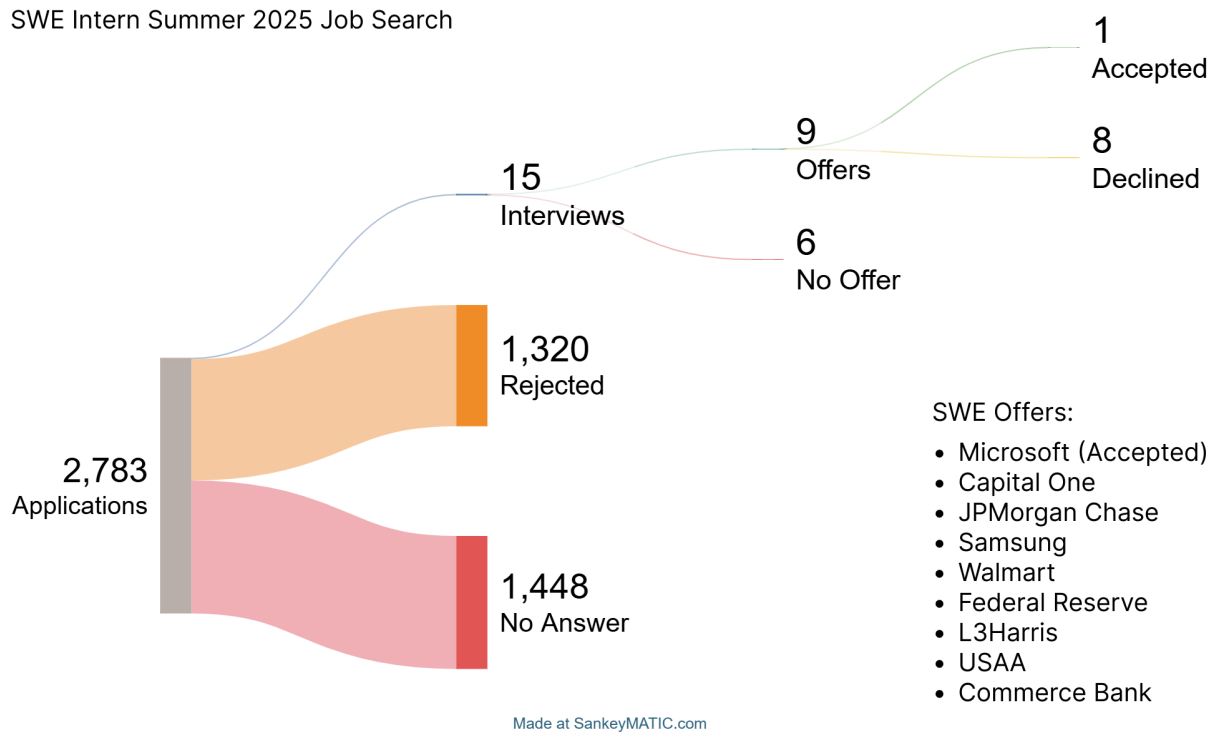
Why I Think Companies Selected Me

Summing up all the things I've done to try and land an internship can be described as just applying and having a good resume. But so many other students can do this as well but they aren't being chosen for these positions. Why me? I think a major factor to why I seem to have an edge over other candidates is because of the technologies I focus on in my projects. From keeping up to date on latest tech and upcoming trends, it is important that I learn these to stay

relevant in the field. This is why in my projects I try to use trending tech like cloud and AI/ML to appeal to the recruiters that are mostly hiring for modern engineers with modern tech knowledge. This is the main thing that will separate a winning resume from a losing resume.

Final Thoughts

If you follow everything in this guide you will have a really high probability of becoming the software engineer you strive to be. Be the person that goes the extra mile. Do more than the other CS students around you. Use doomers and gloomers around you to motivate you to be better than them. Every time you see a doomer CS post on reddit just think to yourself: “if I don’t put in more effort I’ll end up like this person.” That’s what I did at least. Just remember, even the little things help out. I wish you all good luck in your SWE careers!

Appendix

Summer 2025 Sankey Diagram